

Architecture Justification

Student Club Event Ticketing and Budget Portal

Shishir Hegde | SRN PES1UG24CS438 | Section 5H

Architecture selection

I chose a layered architecture for the Student Club Event Ticketing and Budget Portal. The browser handles the forms and display, the application handles the event rules, and a database stores the records. The prototype runs as one Flask application with SQLite. It is also client-server in deployment: browsers send requests to one application server.

Reasons for the choice

First, the portal has clear groups of work: proposals, approvals, tickets and check-in. Keeping these rules separate from the pages makes it easier to change a form without changing the approval process. It also makes the important rules easier to test.

Second, this is a small individual project. A single application is easier to run and debug than several services. Approval records and budget reservations can share one database transaction, so a half-completed request is less likely to leave inconsistent data.

Security advantage

The browser cannot directly update the database. The application checks the logged-in role before changing a request or validating a ticket. Signed QR data makes changes to the ticket detectable, and a conditional database update prevents a second entry with the same ticket.

Performance benefit and limitation

The business components call each other locally, avoiding extra network calls between services. Indexed ticket lookups keep validation simple. This does not guarantee the required latency: SQLite serialises writes, and the 500-user target needs load testing. The local measurements are reported in the testing folder.